c0rrupt

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

We found this file. Recover the flag.

💡 Hints

- Try fixing the file header

🧰 Tools Used

- `file` – Identify file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `xxd` – View and edit hexadecimal representation of files
- `hexedit` – Interactive hexadecimal editor
- **Image viewer** – Display the final image

🔍 Complete Solution

Step 1: Download and Prepare

Download the file named `mystery` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file mystery
```

Expected output:

```
mystery: data
```

The `file` command doesn't recognize the file type – it's corrupted or has a missing/incorrect header.

Step 3: Search for Visible Text

Check if any readable text reveals the flag:

```
strings mystery | grep -i "picoCTF"
```

Output: (no results)

No flag yet.

Step 4: Examine Metadata

```
exiftool mystery
```

Output:

```
ExifTool Version Number         : 13.50
File Name                       : mystery
Directory                       : .
File Size                       : 203 kB
File Modification Date/Time     : 2026:05:16 15:39:12+02:00
File Access Date/Time           : 2026:05:16 15:39:38+02:00
File Inode Change Date/Time     : 2026:05:16 15:39:18+02:00
File Permissions                : -rw-rw-rw-
Error                           : Unknown file type
```

No useful information.

Step 5: Hexdump Analysis

Use `xxd` to view the hexadecimal representation of the file:

```
xxd mystery | head
```

Output:

```
00000000: 8965 4e34 0d0a b0aa 0000 000d 4322 4452  .eN4........C"DR
00000010: 0000 066a 0000 0447 0802 0000 007c 8bab  ...j...G.....|..
00000020: 7800 0000 0173 5247 4200 aece 1ce9 0000  x....sRGB.......
00000030: 0004 6741 4d41 0000 b18f 0bfc 6105 0000  ..gAMA......a...
00000040: 0009 7048 5973 aa00 1625 0000 1625 0149  ..pHYs...%...%.I
00000050: 5224 f0aa aaff a5ab 4445 5478 5eec bd3f  R$......DETx^..?
00000060: 8e64 cd71 bd2d 8b20 2080 9041 8302 08d0  .d.q.-.  ..A....
00000070: f9ed 40a0 f36e 407b 9023 8f1e d720 8b3e  ..@..n@{.#... .>
00000080: b7c1 0d70 0374 b503 ae41 6bf8 bea8 fbdc  ...p.t...Ak.....
00000090: 3e7d 2a22 336f de5b 55dd 3d3d f920 9188  >}*"3o.[U.==. ..
```

Step 6: Identify the Correct File Format

Image files have specific **magic bytes** (file signatures) at the beginning. Common signatures include:

Format	Magic Bytes (Hexadecimal)	ASCII Representation
PNG	`89 50 4E 47 0D 0A 1A 0A`	`‰PNG\r\n\x1a\n`
JPEG	`FF D8 FF`	`ÿØÿ`
JPEG (JFIF)	`FF D8 FF E0`	`ÿØÿà`
JPEG (EXIF)	`FF D8 FF E1`	`ÿØÿá`
GIF	`47 49 46 38`	`GIF8`
BMP	`42 4D`	`BM`

Format	Magic Bytes (Hexadecimal)	ASCII Representation
TIFF (big-endian)	`4D 4D 00 2A`	`MM\x00*`
TIFF (little-endian)	`49 49 2A 00`	`II*\x00`
PDF	`25 50 44 46`	`%PDF`
ZIP	`50 4B 03 04`	`PK\x03\x04`
ELF	`7F 45 4C 46`	`\x7fELF`

Looking at our hexdump, the first byte is `89` – this matches the **PNG** signature! However, the next bytes should be `50 4E 47` ("PNG"), but we see `65 4E 34` instead.

Step 7: Understand the Corruption

Let's analyze what's wrong:

Correct PNG header (8 bytes):

```
89 50 4E 47 0D 0A 1A 0A
```

Current corrupted header:

```
89 65 4E 34 0D 0A B0 AA
```

Offset	Correct	Current	Issue
0	`89`	`89`	✅ Correct
1	`50`	`65`	❌ Should be `50` ('P')
2	`4E`	`4E`	✅ Correct ('N')
3	`47`	`34`	❌ Should be `47` ('G')
4	`0D`	`0D`	✅ Correct
5	`0A`	`0A`	✅ Correct

Offset	Correct	Current	Issue
6	`1A`	`B0`	❌ Should be `1A`
7	`0A`	`AA`	❌ Should be `0A`

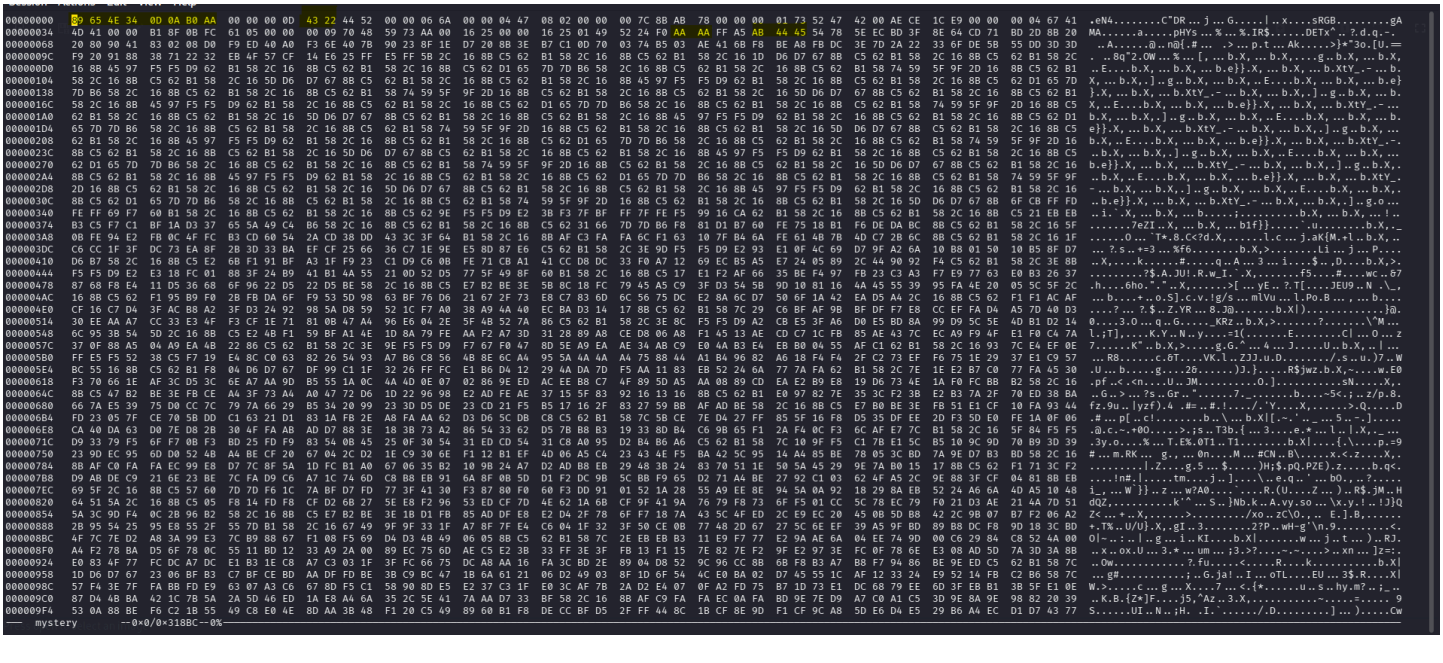
Also, after the header, we expect the chunk structure. At offset 12 (after 8-byte header + 4-byte chunk length), we should see `49 48 44 52` ("IHDR"). Currently, we see `43 22 44 52`.

Step 8: Fix the Corrupted Header

We need to correct the PNG signature and the IHDR chunk identifier.

Use `hexedit` to modify the file:

```
hexedit mystery
```



Navigate to the following offsets and make these changes:

Offset	Original	Correct	Chunk
1	`65`	`50`	PNG signature
3	`34`	`47`	PNG signature
6	`B0`	`1A`	PNG signature

Offset	Original	Correct	Chunk
7	AA	0A	PNG signature
12	43	49	IHDR signature
13	22	48	IHDR signature
14	44	44	IHDR signature (already ‘D’)
15	52	52	IHDR signature (already ‘R’)

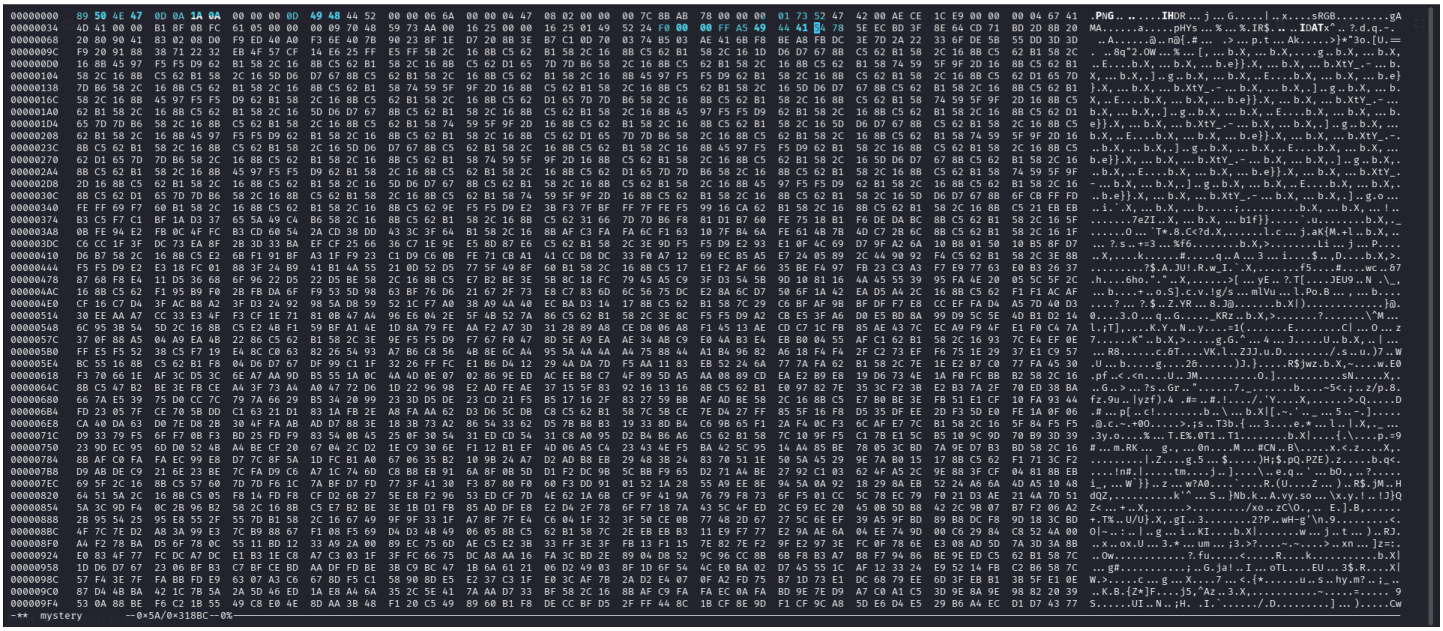
After fixing the header (offset 0-15):

Before:

```
00000000: 8965 4e34 0d0a b0aa 0000 000d 4322 4452  .eN4........C"DR
```

After:

```
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452  .PNG........IHDR
```



Step 9: Fix the IDAT Chunk Corruption

Looking further at offset `0x50` (decimal 80), we see:

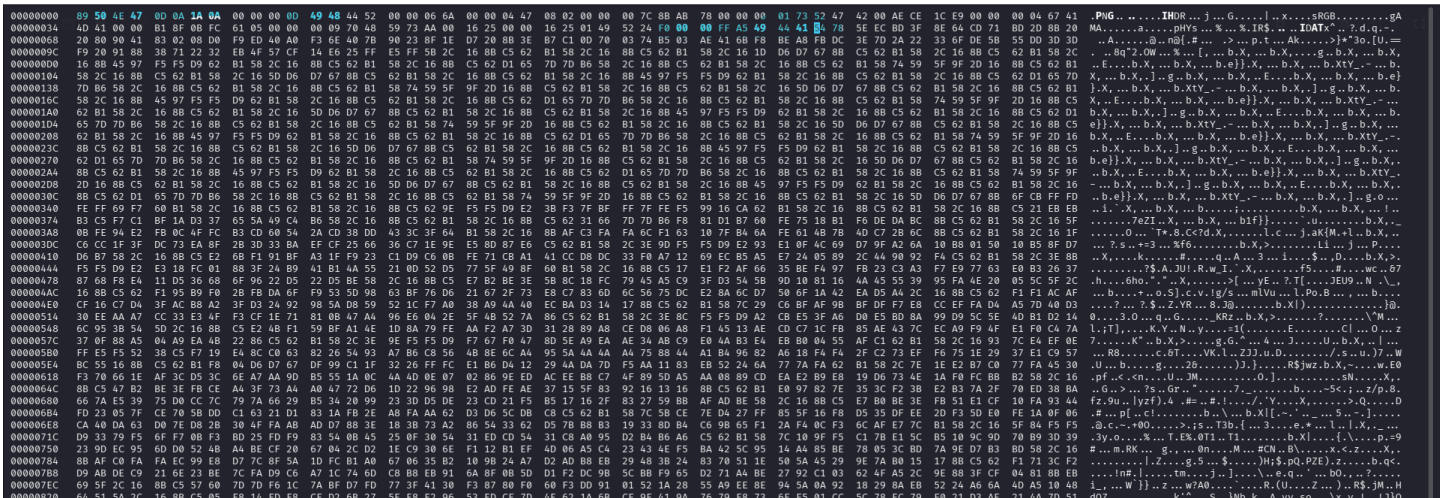
```
00000050: 5224 f0aa aaff a5ab 4445 5478 5eec bd3f  R$......DETx^..?
```

The string `DETx` should actually be `IDAT` (the PNG image data chunk). Let's fix that:

Offset	Original	Correct
0x54	`44` (`D`)	`49` (`I`)
0x55	`45` (`E`)	`44` (`D`)
0x56	`54` (`T`)	`41` (`A`)
0x57	`78` (`x`)	`54` (`T`)

After fixing:

```
00000050: 5224 f000 00ff a549 4441 5478 5eec bd3f  R$.....IDATx^..?
```



> **Note**: The `pHYs` chunk at offset `0x40` was already correct, so no changes were needed there.

Step 10: Save and Verify

After saving the changes, verify the file type:

```
file mystery
```

Output:

```
mystery: PNG image data, 1642 x 1095, 8-bit/color RGB, non-interlaced
```

Check metadata:

```
exiftool mystery
```

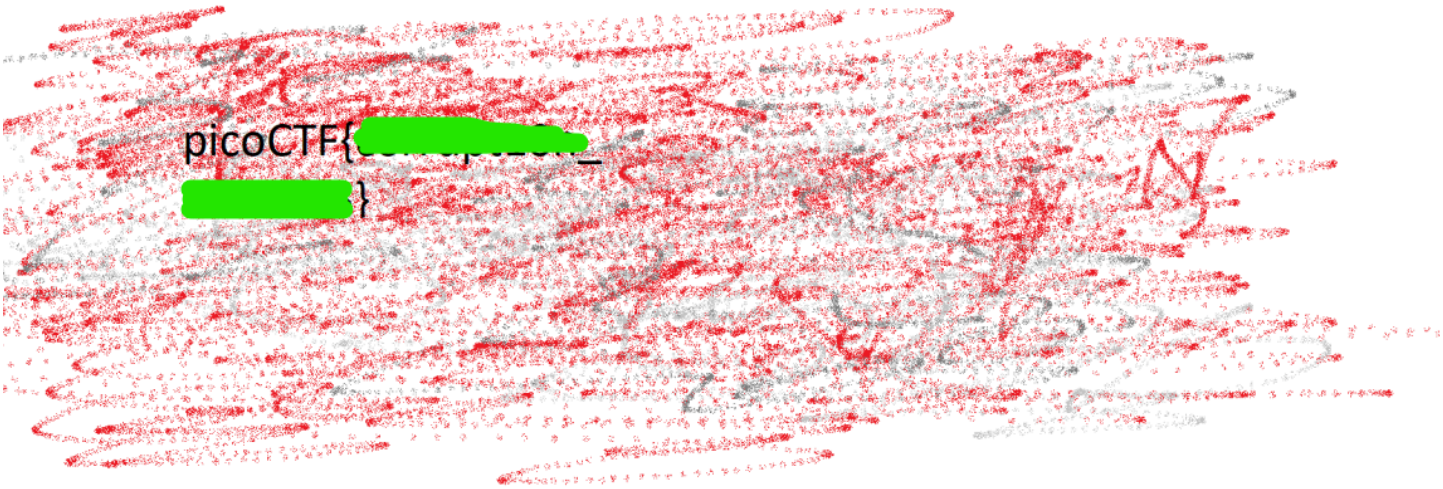
Output:

```
ExifTool Version Number         : 13.50
File Name                       : mystery
Directory                       : .
File Size                       : 203 kB
File Modification Date/Time     : 2026:05:17 08:34:23+02:00
File Access Date/Time           : 2026:05:17 08:34:23+02:00
File Inode Change Date/Time     : 2026:05:17 08:34:23+02:00
File Permissions                : -rw-rw-rw-
File Type                       : PNG
File Type Extension             : png
MIME Type                       : image/png
Image Width                     : 1642
Image Height                    : 1095
Bit Depth                       : 8
Color Type                      : RGB
Compression                     : Deflate/Inflate
Filter                          : Adaptive
Interlace                       : Noninterlaced
SRGB Rendering                  : Perceptual
Gamma                           : 2.2
Pixels Per Unit X               : 2852132389
Pixels Per Unit Y               : 5669
Pixel Units                     : meters
Image Size                      : 1642x1095
Megapixels                      : 1.8
```

Step 11: Open the Image

The file is now a valid PNG. Open it with an image viewer:

```
xdg-open mystery    # Linux
open mystery        # macOS
start mystery       # Windows
```



The flag is visible in the image!

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, opening the repaired PNG image will reveal the complete flag.

🧠 Key Concepts

Concept	Description
Magic Bytes (File Signatures)	Unique byte sequences at the start of a file that identify its format.
PNG Structure	8-byte signature (`%PNG\r\n\x1a\n`) followed by chunks (IHDR, IDAT, IEND, etc.).
PNG Chunks	Each chunk has: 4-byte length, 4-byte type, data, 4-byte CRC.
IHDR Chunk	The first chunk after the PNG signature, contains image dimensions and metadata.
IDAT Chunk	Contains the actual compressed image data.
xxd	Tool to view and manipulate hexadecimal representations of files.
hexedit	Interactive hexadecimal editor for direct file modification.
Corruption Repair	Fixing corrupted file headers by comparing with known magic bytes.

💡 Pro Tips

1. **Always start with** `file` – It's the fastest way to check if a file is recognized.
2. **Use** `xxd | head` **to view the header** – The first 64 bytes often reveal the file type or the corruption pattern.
3. **Memorize common magic bytes** – Knowing PNG (`89 50 4E 47`), JPEG (`FF D8 FF`), and PDF (`25 50 44 46`) is essential for forensics challenges.
4. **Compare with a known good file** – Create a dummy PNG file and compare its hexdump with the corrupted one to identify differences.
5. **Use** `hexedit` **for precision** – It's easier than `xxd` + manual conversion for making multiple edits.
6. **Check for "IDAT" vs "DETx" patterns** – When you see `DETx` , it's almost certainly a corrupted `IDAT` chunk.

7. **Don’t ignore the CRC** – After fixing the header, the CRC may be wrong, but the image might still display. For this challenge, it wasn’t necessary to fix.